Guía 2 de 3 — Logging, pruebas con pytest, configuración externa y refactor con IA

DPL1046 · Clase 7 · Para repasar en casa, a tu ritmo.

Un programa confiable **cuenta lo que hace** (logging), **se prueba solo** (pytest) y **no tiene parámetros escondidos en el código** (configuración externa). Esta guía repasa las tres ideas y suma cómo ordenar tu código con la IA de Copilot, usando los tests como red de seguridad.

> Antes de empezar, haz tu rutina de inicio (Guía 1 de la Clase 2) con el `.venv` activado. Las bibliotecas nuevas de esta clase (`pytest`, `pyyaml`) se instalan con `pip install -r requirements.txt`.

---

1. Logging: el diario de a bordo

Un `print` es **hablar en voz alta**: lo oyes solo si estás ahí. Un **log** es el **diario de a bordo**: queda escrito, con fecha y hora, y lo puedes leer mañana. Es la diferencia entre adivinar qué falló y leerlo.

```
import logging
import pandas as pd

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s",
    handlers=[
        logging.FileHandler("data/pipeline.log", encoding="utf-8"),
        logging.StreamHandler(),
    ],
)

logging.info("Inicio del programa")
orders = pd.read_csv("data/raw/olist_orders_dataset.csv")
logging.info("Pedidos leídos: %s", len(orders))

sin_entrega = orders["order_delivered_customer_date"].isna().sum()
logging.warning("Pedidos sin fecha de entrega: %s", sin_entrega)

try:
    pd.read_csv("data/raw/no_existe.csv")
except FileNotFoundError:
    logging.error("No se encontró el archivo no_existe.csv")
logging.info("Fin del programa")
```

Configuración (`basicConfig`)

Parámetro	Qué hace
`level=logging.INFO`	Desde qué nivel de importancia se anota
`format=...`	Cómo se ve cada línea: hora, nivel y mensaje
`handlers=[...]`	**Dónde** se escribe: `FileHandler` (archivo) y `StreamHandler` (pantalla)

Una línea del log se ve así: `2026-10-27 10:15:02 | INFO | Inicio del programa`.

Niveles (de menos a más grave)

Nivel	Cuándo usarlo
`DEBUG`	Detalle para depurar
`INFO`	"Todo normal, esto pasó"
`WARNING`	Algo raro, pero el programa sigue
`ERROR`	Algo **falló**

El `%s` es un **hueco** que se rellena con el valor que viene después de la coma. Abre `data/pipeline.log` después de ejecutar: verás las mismas líneas guardadas.

---

2. Pruebas automáticas con pytest

Probar una **bicicleta antes de salir**: frenos, luces, cadena. Una prueba automática es lo mismo para tu código: una verificación que se ejecuta sola. Cuando cambies algo, la corres y sabes **en segundos** si todo sigue funcionando. La rúbrica del proyecto pide al menos 3 por función de transformación.

Primero, las funciones a probar, guardadas en su propio archivo (`utilidades.py`):

```python
import pandas as pd

def calcular_dias(compra, entrega):
    """Devuelve los días que pasan entre dos fechas (escritas como texto)."""
    return (pd.to_datetime(entrega) - pd.to_datetime(compra)).days

def clasificar_entrega(dias):
    """Clasifica una entrega según los días que tardó."""
    if dias < 0:
        raise ValueError("Los días no pueden ser negativos")
    if dias <= 7:
        return "rápida"
    elif dias <= 20:
        return "normal"
    else:
        return "lenta"
```

Luego las pruebas (`test_utilidades.py`):

```python
import pytest
from utilidades import calcular_dias, clasificar_entrega

def test_calcular_dias():
    assert calcular_dias("2017-10-02", "2017-10-10") == 8

def test_entrega_rapida():
    assert clasificar_entrega(5) == "rápida"

def test_entrega_normal():
    assert clasificar_entrega(15) == "normal"

def test_entrega_lenta():
    assert clasificar_entrega(45) == "lenta"

def test_dias_negativos_da_error():
    with pytest.raises(ValueError):
        clasificar_entrega(-1)
```

- Una prueba es una **función cuyo nombre empieza con `test_`**: así pytest la encuentra sola.
- `assert` significa **"afirmo que..."**: si es cierto, la prueba pasa; si no, falla y avisa.
- Se prueba **cada camino** del `if`: 5 → rápida, 15 → normal, 45 → lenta.

- `pytest.raises(ValueError)` verifica que, ante un dato absurdo, **sí falle** con ese error. También se prueba que el error aparezca cuando debe.
- `raise ValueError(...)` es como el programa **avisa** que algo anda mal a propósito.

Para ejecutarlas: `pytest clases/clase-07/material/ -v`. Cada prueba muestra `PASSED` (verde) o `FAILED` (rojo).

> **Truco para entender:** cambia `== "rápida"` por `== "lenta"`, corre pytest, mira la prueba fallar en rojo, y luego devuélvelo.

---

3. Configuración externa

> **Esta parte es opcional en clase:** el script `99_opcional_config.py` la practica en casa, y la verás en uso en `config.yaml` del proyecto de ejemplo de la Clase 8.

Un **termostato**: la temperatura que quieres la cambias en el dial, no abres la caldera a modificar los cables. Los números que se ajustan (carpetas, umbrales) van en un archivo aparte, `config.yaml`; el código no se toca.

`config.yaml`:

```yaml
datos:
  carpeta: "data/raw/"
  pedidos: "olist_orders_dataset.csv"
umbral_dias_lento: 20
```

Y el script que lo lee:

```python
import yaml
import pandas as pd

with open("clases/clase-07/material/config.yaml", encoding="utf-8") as archivo:
    config = yaml.safe_load(archivo)

ruta = config["datos"]["carpeta"] + config["datos"]["pedidos"]
umbral = config["umbral_dias_lento"]

orders = pd.read_csv(ruta)
compra = pd.to_datetime(orders["order_purchase_timestamp"])
entrega = pd.to_datetime(orders["order_delivered_customer_date"])
dias = (entrega - compra).dt.days
print("Entregas lentas (más de", umbral, "días):", (dias > umbral).sum())
```

- `with open(...) as archivo:` abre el archivo y lo **cierra solo** al terminar. `encoding="utf-8"` evita problemas con las tildes.
- `yaml.safe_load(archivo)` convierte el YAML en un **diccionario** (Clase 3).
- `config["datos"]["carpeta"]` es un acceso anidado, como en JSON.
- **Prueba:** cambia `umbral_dias_lento` a `10` en el YAML y vuelve a correr, **sin tocar el `.py`**: el resultado cambia.

Archivo	Para qué	¿Se sube a Git?
`config.yaml`	Parámetros (rutas, umbrales)	Sí
`.env`	**Secretos** (claves)	**No**

---

4. Refactor con la IA de Copilot en VS Code

Refactor es reescribir el código para que se lea mejor (nombres claros, sin repetir, con docstring) **sin cambiar lo que hace**. Copilot te ayuda a proponerlo; tus **tests** comprueban que no se rompió nada.

Antes de empezar: instala la extensión **GitHub Copilot** en VS Code (el plan gratuito basta) e inicia sesión con tu cuenta de GitHub.

Paso a paso con `04_refactor_copilot.py`

1. Ejecuta los tests de antes, desde la raíz del repo y con `(.venv)` activo:

```
pytest clases/clase-07/material/ -v
```

Deben pasar los 9 tests.

2. Abre `04_refactor_copilot.py` y **selecciona la función** `resumir_pedidos` completa.
3. Abre el chat en línea: **Cmd+I** (Mac) o **Ctrl+I** (Windows) y escribe:

```
Refactoriza esta función: usa nombres descriptivos, evita repetir el
recorrido de la lista y agrega un docstring. No cambies el nombre de la
función ni lo que devuelve.
```

4. Mira el cambio que propone Copilot. Acéptalo solo si lo entiendes.
5. Vuelve a ejecutar los tests. Deben seguir los 9 `PASSED`.

> Si no puedes usar Copilot, hazlo a mano: cambia los nombres `p`, `t`, `c`, `l`, `a`, `x` por nombres claros y vuelve a correr los tests.

Cómo revisar lo que propone la IA

Pregunta	Qué haces
¿Hace lo mismo?	Ejecuta `pytest`: deben seguir pasando todos

Pregunta	Qué haces
¿La entiendo?	Lee cada línea; si no, pídele a Copilot que la explique
¿Cambió algo que no pedí?	Compara el antes y el después
¿Qué le estoy mostrando?	**Nunca** pegues claves ni datos personales en el chat

Para tu proyecto: refactoriza **una** función, con sus tests en verde antes y después.

Errores frecuentes

Qué ves	Por qué pasa	Qué hacer
`ModuleNotFoundError: pytest` / `yaml`	Falta instalar	Activa el `.venv` y `pip install -r requirements.txt`
`no tests ran`	El archivo o la función no empiezan con `test_`	Renómbralos
`ModuleNotFoundError: utilidades`	pytest no encuentra el archivo	Ejecuta pytest apuntando a la carpeta del material
El log no se crea	No existe la carpeta `data/`	Crea la carpeta o cambia la ruta
`yaml.scanner.ScannerError`	Sangría o dos puntos mal en el YAML	En YAML la sangría es de **espacios** (no tabulaciones)
`KeyError` al leer la config	La clave no existe en el YAML	Compara con `print(config)`

Practica tú

1. ¿Qué nivel usarías para avisar "no encontré el archivo"? ¿Y para "pedidos leídos: 99441"?
2. Escribe una prueba para `clasificar_entrega(20)`. ¿Qué debe devolver?
3. ¿Dónde guardarías un `umbral_dias_lento` y dónde una clave de API?

▶ Ver respuestas

Para llevarte

- **Logging** = diario de a bordo: queda escrito, con hora y nivel.
- **pytest**: funciones `test_...` con `assert`; se prueba cada camino y también los errores.
- **Config externa**: los parámetros en `config.yaml`, los secretos en `.env`.
- **Refactor con IA**: primero tests, después Copilot; revisa el cambio y vuelve a correr los tests.